翁恺课 · 第 7 主题

W7 数组运算

对齐翁恺《程序设计入门——C语言》第 7 讲 · 目标：会线性查找与二分查找、会写选择排序与冒泡排序、认识二维数组（行×列）并用双重循环遍历 · 不覆盖：指针与字符串（W8）

7.1 数组运算"三板斧"：查找 / 排序 / 统计

数组存下一批数据后，最常做的三件事是：**查找**（某数在不在、在哪）、**排序**（从小到大/从大到小）、**统计**（求和、最大、个数——W5 已经会）。本讲重点攻前两件，翁恺的说法：**查找和排序是数组的"两道必过关卡"**，几乎所有算法题都绕不开。

> **一句话：**查找分**线性**（挨个扫，O(n)，不要求有序）和**二分**（折半，O(log n)，要求有序）；排序先学会**选择**和**冒泡**这两种最直观的。

7.2 线性查找：挨个问"是你吗"

最简单也最通用的查找：从 `a[0]` 一路问到 `a[n-1]`，找到就返回位置。数据**不用有序**。找不到时返回一个**不可能的下标**（惯例用 `-1`）：

```
/* wk7_search.c — 线性查找：在数组里找目标值的位置（gcc 实跑）*/
#include <stdio.h>

int find(int a[], int n, int target);

int main(void)
{
    int a[6] = {3, 9, 1, 12, 5, 8};
    int pos = find(a, 6, 12);
    if (pos >= 0)
        printf("找到 12，位置是 a[%d]\n", pos);
    else
        printf("没找到\n");
    return 0;
}

int find(int a[], int n, int target)
{
    int i;
    for (i = 0; i < n; i++) {
        if (a[i] == target) return i;
    }
```

```
    return -1;   /* 找不到，返回 -1 */
}
```

实跑输出：

```
找到 12，位置是 a[3]
```

> **新手坑：拿 0 当"找不到"。**数组下标从 0 开始，`a[0]` 是合法位置。如果函数返回 0 表示"找不到"，那 `find` 真的找到 `a[0]` 时你会误判成没找到。惯例用 **-1**（不可能的下标）。

7.3 选择排序：每轮"挑最小的放前面"

选择排序的思路很直白：**第一轮从整个数组挑最小，放位置 0；第二轮从剩下的挑最小，放位置 1**……像排队时不停把"最矮的"拉出来排到前面。找一个叫"擂主位置"的变量 `min` 记下标：

```
/* wk7_sort.c — 选择排序：升序（gcc 实跑）*/
#include <stdio.h>

void print_array(int a[], int n);

int main(void)
{
    int a[6] = {5, 2, 8, 1, 9, 3};
    int i, j, min, tmp;
    for (i = 0; i < 5; i++) {      /* 前 n-1 个位置 */
        min = i;                   /* 假设 i 是最小位置 */
        for (j = i + 1; j < 6; j++) {
            if (a[j] < a[min]) min = j;  /* 找最小 */
        }
        tmp = a[i]; a[i] = a[min]; a[min] = tmp;  /* 换到 i */
    }
    print_array(a, 6);
    return 0;
}

void print_array(int a[], int n)
{
    int i;
    for (i = 0; i < n; i++) printf("%d ", a[i]);
    printf("\n");
}
```

对 {5,2,8,1,9,3}，实跑输出：

```
1 2 3 5 8 9
```



图 W7-3　选择排序：第一轮找出全局最小（绿色 1），与位置 0 交换。每轮锁定一个"已归位"位置，n-1 轮完成

7.4 冒泡排序：相邻"打擂台"换位置

冒泡排序是另一种直观思路：**从头两两比较，大的往后冒**——像气泡往上浮。每一趟从前往后，如果前一个比后一个大就交换，一趟下来**最大的被"冒"到最后**；下一趟少比一个（最后已归位）：

```
/* wk7_bubble.c — 冒泡排序：升序（gcc 实跑）*/
#include <stdio.h>

void print_array(int a[], int n);

int main(void)
{
    int a[6] = {5, 2, 8, 1, 9, 3};
    int i, j, tmp;
    for (i = 0; i < 5; i++) {        /* 趟数 */
        for (j = 0; j < 5 - i; j++) { /* 每趟少比一个 */
            if (a[j] > a[j + 1]) {
                tmp = a[j]; a[j] = a[j + 1]; a[j + 1] = tmp;
            }
        }
    }
    print_array(a, 6);
    return 0;
}

void print_array(int a[], int n)
{
    int i;
```

```
    for (i = 0; i < n; i++) printf("%d ", a[i]);
    printf("\n");
}
```

对 {5,2,8,1,9,3}，实跑输出：

```
1 2 3 5 8 9
```

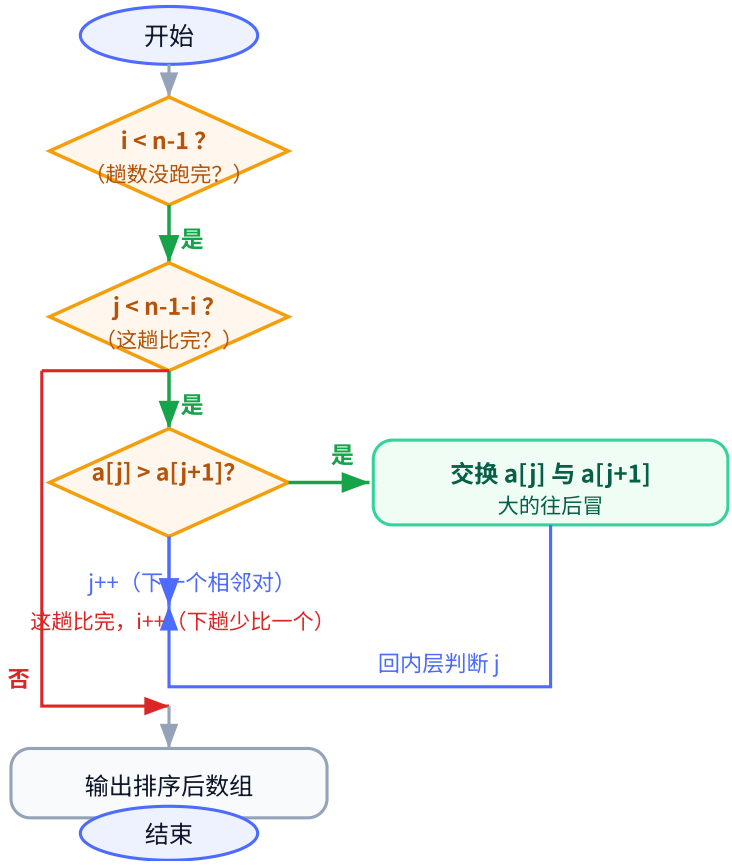


图 W7-2　冒泡排序流程：外层管趟数，内层两两比较，大的往后冒；每趟末尾一个归位，下一趟少比一个

7.5 二分查找：有序数组里"折半找"

如果数组**已经有序**（升序或降序），就不用挨个扫了——**二分查找**每次砍掉一半：看中间那个数，比目标小就去右半边，比目标大就去左半边，直到锁定或区间为空。这就是 O(log n) 的威力（10 亿个数也只要约 30 次）：

```
/* wk7_bin.c — 二分查找：有序数组里找目标（gcc 实跑）*/
#include <stdio.h>

int bin_search(int a[], int n, int target);

int main(void)
{
```

```
    int a[7] = {1, 3, 5, 7, 9, 11, 13};   /* 必须有序 */
    int pos = bin_search(a, 7, 9);
    if (pos >= 0)
        printf("找到 9，位置是 a[%d]\n", pos);
    else
        printf("没找到\n");
    return 0;
}

int bin_search(int a[], int n, int target)
{
    int lo = 0, hi = n - 1, mid;
    while (lo <= hi) {
        mid = (lo + hi) / 2;
        if (a[mid] == target) return mid;
        else if (a[mid] < target) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1;
}
```

实跑输出：

```
找到 9，位置是 a[4]
```



图 W7-1　线性查找 vs 二分查找：线性 O(n) 不要求有序、挨个扫；二分 O(log n) 快得多但必须先排序

7.6 二维数组初识：一张"行×列"的表格

一维数组是一排盒子，**二维数组就是一张表格**：行 × 列，用两个下标访问 `m[行][列]`。最常见的是存矩阵、棋盘、成绩表（每行一个学生、每列一个科目）。遍历要用**双重循环**——外层管行、内层管列：

```
/* wk7_matrix.c — 二维数组：3x3 矩阵求和（gcc 实跑）*/
#include <stdio.h>

int main(void)
{
    int m[3][3] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
    int i, j, sum = 0;
    for (i = 0; i < 3; i++) {
        for (j = 0; j < 3; j++) {
            sum += m[i][j];
        }
    }
    printf("3x3 矩阵元素之和 = %d\n", sum);
    return 0;
}
```

实跑输出：

```
3x3 矩阵元素之和 = 45
```

> **翁恺提醒：**二维数组遍历记住"**外层是行、内层是列**"（先固定一行扫完所有列，再换下一行）。学完一维的下标从 0 开始，二维也一样：`m[3][3]` 行下标 0~2、列下标 0~2。

7.7 新手坑集中营（本讲最容易错的 3 件事）

1. **用 0 当"找不到"** → 数组下标从 0 开始，`a[0]` 是合法位置。查找返回"找不到"要用 `-1`，否则找到了 `a[0]` 会误判。
2. **二分查找的前提忘排序** → 二分只对**有序数组**有效。无序数组二分会得出**错误结果**。用二分前要么确认已有序，要么先排序。
3. **冒泡内层边界写错（越界）** → 内层该写 `j < n-1-i`，写成 `j < n-1` 会让 `a[j+1]` 访问到越界。gcc 实测**不报**（未定义行为），而且运行**常常"碰巧对"**（见练习册改错④）——排序类逻辑错编译器不帮你，只能自己推演边界。

7.8 一分钟自测（答案见练习册）

判断/填空：

1. 线性查找返回 -1 表示____；为什么不用 0 表示"找不到"？因为 ____。
2. 二分查找的时间复杂度是 ____，它要求数组必须先 ____。

3. 冒泡排序内层循环条件应写 `j < ____`（n 为数组大小）；二维数组 `m[3][3]` 的列下标范围是 ____ 到 ____。

7.9 练习指引

去配套练习册做 W7 练习。翁恺建议本讲里程碑：**把一组成绩用排序排好，输出名次表**——把"查找 + 排序"真正用起来。这之后你对数组的理解，就足以进入 W8 指针的世界了。

本讲义为自写原创讲解（对齐翁恺《程序设计入门——C语言》第 7 讲），仅作课堂伴读；示例代码均用 gcc -Wall -Wextra 实测通过（MinGW-W64 gcc 16.1.0），输出为真实运行结果（含 gcc 警告原文）。视频与图片版权归原作者所有，本讲义不含课程原素材。